Funkce: .sort() a sorted()

Základní rozdíly

list.sort()

Typ: Metoda třídy list

Způsob: In-place (přímo v paměti)

Návratová hodnota: Vždy None

Použití: Pouze pro seznamy (list)

Paměť: Efektivnější pro velké seznamy

sorted()

Typ: Vestavěná funkce (Built-in)

Způsob: Vytváří novou kolekci

Návratová hodnota: Nový seřazený list

Použití: Jakýkoliv iterable (tuple, dict, set)

Paměť: Alokuje paměť pro nový seznam

UKÁZKA #1

Metoda list.sort() v praxi

```
# 1. In-place modifikace původního seznamu
cisla = [5, 2, 8, 1, 3]
vysledek = cisla.sort()
print(cisla) # [1, 2, 3, 5, 8] <- změněno
print(vysledek) # None <- NÁVRATOVÁ HODNOTA!
# 2. Řazení sestupně pomocí reverse=True
cisla.sort(reverse=True)
print(cisla) # [8, 5, 3, 2, 1]
# 3. CHYBA: Volání na jiném typu selže
tup = (3, 1, 2)
tup.sort()
# AttributeError: 'tuple' object has no attribute 'sort'
```

In-Place Mutace

Upravuje data přímo na původní adrese v paměti. Nešetří čas při vytváření nových objektů.

Ppast: NoneType

Přiřazení x = data.sort() přepíše vaši proměnnou na None!

UKÁZKA #2

Funkce sorted() a libovolné Iterables

```python
# 1. Původní objekt zůstává nezměněn
original = [5, 2, 8, 1]
serazene = sorted(original)
print(original) # [5, 2, 8, 1] <- Původní zachován
print(serazene) # [1, 2, 8, 5] <- Nový list
# 2. Funguje na Tuple, String, Set i Dict
slovo = "python"
print(sorted(slovo)) # ['h', 'n', 'o', 'p', 't', 'y']
```

Univerzálnost

Přijímá jakýkoliv iterovatelný objekt (iterable). Vždy ale vrací nový list.

Imutabilní struktury

Ideální pro řazení n-tic (tuple) nebo řetězců, které nelze měnit přímo.

POKROČILÉ PARAMETRY

Transformace pomocí parametru key=

```python
# 1. Case-insensitive řazení textu
slova = ["banán", "Ahoj", "Citrón"]
print(sorted(slova, key=str.lower))
# Output: ['Ahoj', 'banán', 'Citrón']
# 2. Řazení podle délky prvku
print(sorted(slova, key=len))
# Output: ['Ahoj', 'banán', 'Citrón']
# 3. Řazení slovníků podle hodnoty (operator.itemgetter)
from operator import itemgetter
data = [{"jmeno": "Petr", "vek": 30}, {"jmeno": "Eva", "vek": 22}]
serazeno_dle_veku = sorted(data, key=itemgetter("vek")) # Eva (22) bude před Petrem (30)
```

Jak funguje key?

Funkce zadaná v key se zavolá přesně **jednou pro každý prvek** před samotným porovnáváním.

Výkonnost

operator.itemgetter nebo attrgetter jsou v C-Pythonu rychlejší než lambda funkce.

SHRNUTÍ

Kdy co použít?

Použijte .sort() pokud:

Pracujete výhradně se seznamem a chcete ušetřit paměť modifikací stávajícího objektu.

Použijte sorted() pokud:

Řadíte jakýkoliv jiný iterable (tuple, dict) nebo potřebujete zachovat původní data nezměněná.

Děkuji za pozornost